ADAPTIVE DENSITY-OF-STATES SWITCHING FOR PERFORMANCE OPTIMIZATION IN THERMOELECTRIC ENERGY-HARVESTING SYSTEMS: A C++ AND MATLAB COMPUTATIONAL FRAMEWORK

By

Esabu Blessing Obehi
240408054

A Research Paper Submitted in Partial Fulfillment of the Requirements for EEG 226: Physical Electronics

Department of Electrical and Electronics Engineering
University of Lagos
Lagos, Nigeria.

June 2026

Lecturer: Dr. A.E. Ibhaze

Word Count: 6212 words

TABLE OF CONTENTS

Abstract
1. Introduction
2. Literature Review
    2.1 The Thermoelectric Figure of Merit
    2.2 Density-of-States Engineering
    2.3 Computational Approaches to Transport
    2.4 Gap and Positioning
3. Methodology
    3.1 Modelling Assumptions
    3.2 Density-of-States Model
    3.3 Transport Coefficients
    3.4 Harvested-Power Metric
    3.5 Element Bank and Controllers
    3.6 Operating Profiles and Switching Cost
    3.7 Implementation and Validation
4. Results and Discussion
    4.1 No Single Element is Optimal Everywhere
    4.2 Adaptive Gain and the Effect of Volatility
    4.3 Robustness Against Unknown Conditions
    4.4 The Effect of Switching Cost
    4.5 Synthesis
5. Conclusion
6. Limitations and Future Work
References
Appendix A: Source Code Listings

Abstract

Thermoelectric generators convert heat directly into electricity, but a given thermoelectric material is optimised for only a narrow temperature window because its electronic density of states (DOS) is fixed. In real applications, the operating temperature is rarely steady, so a fixed material spends much of its time away from its optimal point and harvests less energy than it could. This work investigates an adaptive alternative: a bank of thermoelectric elements, each tuned to a different temperature range through DOS engineering, together with a controller that selects the most efficient element for the prevailing condition. A computational framework was developed to test whether adaptive switching outperforms any single fixed element. The transport physics was implemented in C++ as a fast numerical core under the constant relaxation-time approximation, and independently re-implemented in MATLAB for cross-validation; the two models agree to five significant figures on deterministic test points. Using an efficiency-based harvested-power metric derived from the dimensionless figure of merit, the framework was exercised across temperature profiles of varying volatility and against a switching-cost model. The results show that adaptive switching yields a modest but consistent energy advantage over the best fixed element, rising from about 1.3% under smooth operation to roughly 2.0% under highly volatile profiles, and approximately 5% over the average fixed choice across all conditions. A key finding is that the best fixed element changes with the operating profile, so adaptive control provides robustness by removing the need to know the profile in advance. When switching costs are introduced, a naive controller can lose its advantage entirely, but a cost-aware controller with hysteresis preserves a net positive gain across all tested cost levels. The study concludes that adaptive DOS switching is worthwhile in wide, volatile, or unpredictable thermal environments when paired with cost-aware control, while a single

well-chosen element remains preferable for stable conditions. The contribution is a validated, reusable framework and a clear characterisation of the regime in which adaptivity pays off, intended as a foundation for further device-level research.
Keywords: thermoelectric energy harvesting; density of states; adaptive switching; figure of merit; computational modelling; cost-aware control.

1. Introduction

Energy efficiency has become a central concern in modern engineering, and a large fraction of the energy used worldwide is ultimately wasted as heat. Thermoelectric generators offer a way to recover some of this waste by converting a temperature difference directly into electrical power, with no moving parts, no working fluid, and silent, maintenance-free operation (Snyder and Toberer, 2008). These properties make them attractive for waste-heat recovery, remote sensing, wearable electronics powered by body heat, and spacecraft power systems.

The performance of a thermoelectric material is summarised by the dimensionless figure of merit, ZT. A higher ZT means a more efficient material. The figure of merit depends on the material's electronic structure, and in particular on its density of states (DOS), which describes how many electronic states are available at each energy. The shape of the DOS near the Fermi level governs the balance between the three competing properties that determine ZT, namely the Seebeck coefficient, the electrical conductivity, and the thermal conductivity (Mahan and Sofo, 1996).

A practical difficulty is that the DOS of a given material is fixed once it is manufactured, so the material is efficient only within a narrow temperature window. Real operating environments, however, are rarely steady. The temperature of an engine, a sensor node, or a wearable device changes continuously, which means a fixed material is frequently forced to operate away from its optimal point. The energy lost in these off-design conditions is the motivation for this study.

This work asks a simple question: if the temperature changes, can the device adapt its electronic behaviour to match? The proposed approach is an adaptive system consisting of a bank of thermoelectric elements, each engineered through its DOS to be most efficient in a different temperature range, together with a controller that selects the best element for the prevailing temperature. This is analogous to a vehicle changing gears, where each gear is suited to a different speed.

The central aim is to determine whether such adaptive switching harvests more energy than any single fixed element, and to identify the conditions under which it does. The specific objectives are: (i) to develop a validated computational model of a thermoelectric element from its DOS; (ii) to extend the model to a bank of elements with a selection controller; (iii) to quantify the energy advantage of adaptive switching across operating profiles of differing volatility; and (iv) to assess whether this advantage survives the energy cost of switching itself.

The contribution of the paper is twofold. First, it provides a fast, reusable computational framework, implemented in C++ and cross-validated against an independent MATLAB model, for evaluating adaptive thermoelectric strategies. Second, it offers an honest characterisation of when adaptivity is

worthwhile, showing that the benefit is modest but real, that it grows with the volatility of the operating profile, that it provides robustness against unknown future conditions, and that it must be paired with cost-aware control to be preserved in practice. The work is positioned as a foundation for subsequent device-level research rather than as a finished device.

2. Literature Review

2.1 The Thermoelectric Figure of Merit

The efficiency of a thermoelectric material is governed by the figure of merit, ZT, which combines the Seebeck coefficient, the electrical conductivity, the thermal conductivity, and the absolute temperature into a single dimensionless number (Snyder and Toberer, 2008). Improving ZT is difficult because these properties are coupled: increasing the carrier concentration to raise conductivity tends to lower the Seebeck coefficient, and part of the thermal conductivity is carried by the same electrons that carry current. Decades of research have therefore focused on decoupling these properties.

2.2 Density-of-States Engineering

One influential route to higher ZT is to reshape the electronic density of states near the Fermi level. Mahan and Sofo (1996) showed theoretically that a sharply peaked DOS can maximise thermoelectric performance, and Heremans et al. (2008) demonstrated this experimentally by introducing thallium impurity levels into lead telluride, producing a distortion of the density of states that roughly doubled the figure of merit at elevated temperature. The underlying mechanism is captured by the Mott relation, in which the Seebeck coefficient depends on the energy slope of the DOS at the Fermi level (Cutler and Mott, 1969). A steep slope, created by a resonant feature placed just off the Fermi level, enhances the Seebeck coefficient. More recent studies have continued to exploit resonant levels and band-structure engineering in materials such as tin selenide and germanium selenide, confirming that the physics is well established and remains actively used.

2.3 Computational Approaches to Transport

On the modelling side, the dominant workflow combines first-principles band structures with semiclassical Boltzmann transport theory. Tools such as BoltzTraP (Madsen and Singh, 2006) compute transport coefficients from a band structure under the constant relaxation-time approximation, and have been applied to large materials databases. These tools are powerful but operate as forward evaluators: they report the transport properties of a single, fixed

material at a given doping and temperature. They are not designed to search for an optimal DOS shape, nor to model a system whose effective electronic structure changes during operation.

2.4 Gap and Positioning

The literature therefore establishes two facts relevant to this study. First, DOS engineering through resonant features is a proven way to raise ZT in a fixed material. Second, existing computational tools evaluate fixed materials rather than adaptive systems. What has received little attention is the idea of treating the DOS as something that can be switched during operation to track changing conditions, and the question of whether doing so yields a net benefit once the cost of switching is considered. The present work addresses this gap with a lightweight, purpose-built framework and an explicit cost analysis, rather than proposing new materials physics.

3. Methodology

3.1 Modelling Assumptions

The model is deliberately constructed at the level of transport physics rather than first-principles materials simulation, so that many configurations can be evaluated quickly. Three assumptions follow from this choice and are stated explicitly. First, an analytic model DOS is used rather than a density-functional-theory band structure; this makes the framework material-agnostic and fast, at the cost of not representing any specific compound. Second, the constant relaxation-time approximation is adopted for carrier scattering, consistent with established tools such as BoltzTraP (Madsen and Singh, 2006). Third, the adaptive system is modelled as a bank of discrete elements that are switched between, so that each element is an ordinary, physically reasonable material and only the selection is adaptive. The model works in reduced units in which the Boltzmann constant and the electron charge are set to unity; consequently, it validates trends and relative comparisons rather than the absolute properties of a named material.

3.2 Density-of-States Model

Each element is represented by a baseline parabolic-band DOS with an optional resonant peak. The baseline rises as the square root of energy above a band edge, representing a conventional three-dimensional band. The resonant feature is modelled as a Lorentzian peak with three parameters: its position relative to the band, its width, and its height. These three parameters are the tuning knobs through which an element is specialised for a particular temperature range. The total density of states is the sum of the baseline and the resonant contribution.

$$g(E) = A_0\sqrt{(E - E_{edge})} + A \cdot \Gamma^2 / [\,(E - E_0)^2 + \Gamma^2\,] \quad (1)$$

where E_0 is the peak position, Γ its width, A its height, and A_0 the baseline amplitude.

3.3 Transport Coefficients

Transport properties are obtained from the standard moments of the transport distribution function, weighted by the derivative of the Fermi-Dirac occupation. Three moments are computed by numerical integration over energy:

$$L_n = \int \left(-\partial f/\partial E \right) \cdot \Sigma(E) \cdot (E - E_F)^n \, dE \quad (2)$$

Here, f is the Fermi-Dirac distribution, E_F the Fermi level, and Σ(E) the transport distribution, taken as the product of the DOS and a velocity-squared-times-relaxation-time term under the constant relaxation-time approximation. The electrical conductivity, Seebeck coefficient, and electronic thermal conductivity follow from the moments:

$$\sigma = e^2 L_0, \; S = (1/eT)(L_1 / L_0), \; \kappa_e = (1/T)(L_2 - L_1^2 / L_0) \quad (3)$$

The figure of merit combines these with a fixed lattice (phonon) contribution to the thermal conductivity:

$$ZT = S^2 \sigma T / (\kappa_e + \kappa_{lattice}) \quad (4)$$

3.4 Harvested-Power Metric

Comparing elements by raw electrical power is misleading because raw power rises steeply with temperature regardless of material quality, which would trivially favour high-temperature operation. To isolate conversion quality, an efficiency-based harvested-power metric is used, in which the useful output is weighted by an efficiency factor that increases with ZT. This removes the temperature bias and ensures the comparison reflects how well each element converts the heat available at a given temperature, which is the quantity relevant to adaptive selection.

$$P_{harvested} \propto ZT / (ZT + 2) \quad (5)$$

3.5 Element Bank and Controllers

The adaptive system comprises a bank of elements, each defined by a resonant peak centred at a different energy and a corresponding Fermi level, so that each is the most efficient element in a different temperature window. Two controllers are studied. The naive controller selects the instantaneously most efficient element at every time step. The cost-aware controller introduces hysteresis: it switches only when another element's advantage exceeds the energy cost of switching, thereby avoiding wasteful switching for negligible gains. The naive controller represents the simplest possible strategy, while the cost-aware controller represents a realistic strategy that accounts for the penalty of changing elements.

3.6 Operating Profiles and Switching Cost

To emulate real operation, temperature profiles are generated with a controllable volatility parameter ranging from zero, representing a smooth slow swing, to one, representing rapid wide fluctuations, while holding the mean temperature comparable across profiles. For each profile the energy harvested by the adaptive system is compared against the energy harvested by each fixed element operated alone over the same profile. The switching cost is modelled as a per-switch energy penalty expressed as a fraction of the typical per-step harvested energy, so that its magnitude is directly interpretable relative to the gains.

3.7 Implementation and Validation

The framework was implemented in C++ for speed, with the transport integration and switching logic forming the computational core. An independent MATLAB implementation of the same physics was written for cross-validation. Agreement between the two on deterministic test points confirms that the core is correct. Because the volatility and cost studies use random temperature profiles, and the two languages use different random-number generators, the stochastic studies are validated at the level of reproduced trends rather than identical sample values, while the underlying physics is validated deterministically.

Table 1: Full cross-validation grid. The independent C++ and MATLAB cores agree to five significant figures across all bank elements and test temperatures, confirming the framework is correctly implemented in both languages.

T	**Element**	**C++ ZT**	**MATLAB ZT**
0.10	E0	0.08433	0.08433
0.10	E1	0.10059	0.10059
0.10	E2	0.11099	0.11099
0.10	E3	0.10372	0.10372
0.10	E4	0.08336	0.08336
0.30	E0	0.45942	0.45942
0.30	E1	0.50977	0.50977
0.30	E2	0.55074	0.55074
0.30	E3	0.54860	0.54860
0.30	E4	0.48711	0.48711
0.50	E0	1.15631	1.15631
0.50	E1	1.17163	1.17163
0.50	E2	1.16104	1.16104
0.50	E3	1.10178	1.10178
0.50	E4	0.98539	0.98539

4. Results and Discussion

4.1 No Single Element is Optimal Everywhere

The first result establishes the premise of the study. When the figure of merit of a plain element is compared with that of an element carrying a resonant peak, across a range of temperatures, the two curves cross. The resonant element is superior at low temperature, where the narrow transport window sits directly on the sharp DOS feature, while the plain element becomes superior at higher temperature, where the widening transport window averages over the peak and the broader band responds better. This crossover is the physical basis for adaptive switching: because no single element dominates across the whole temperature range, there is room for a system that selects the right element for each condition.

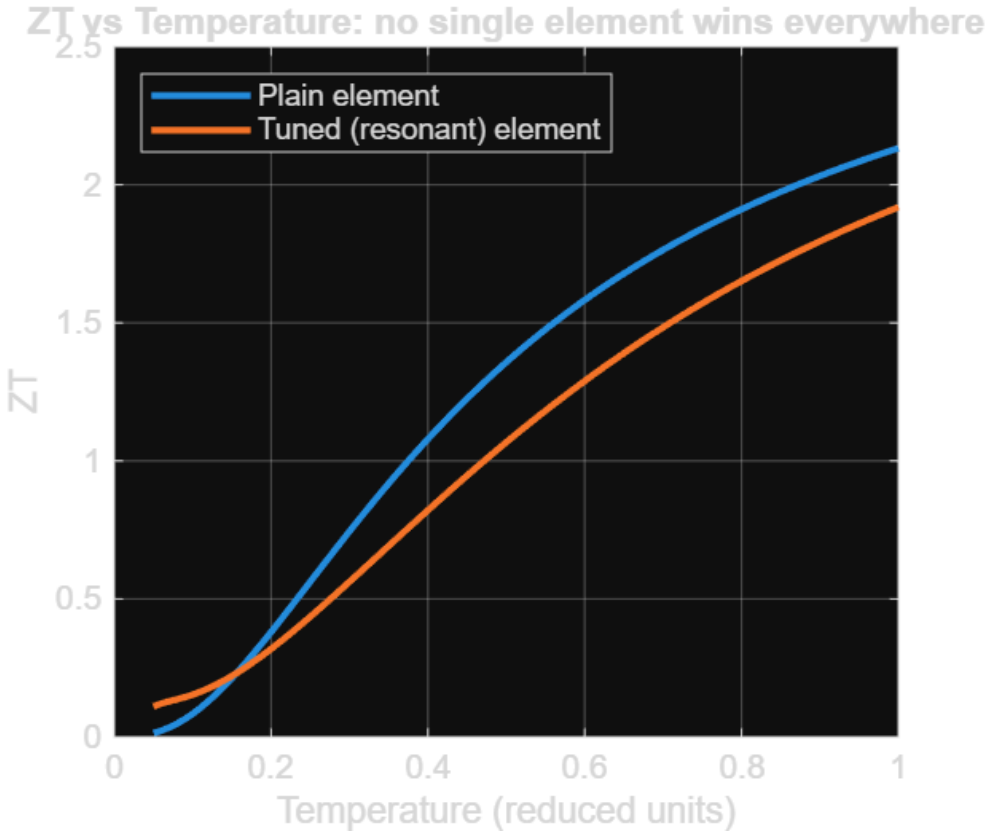


Figure 1: Figure of merit ZT versus temperature for a plain element and a resonant (tuned) element, showing the crossover. (Source: sweep_temp.csv / crossover.png.)

4.2 Adaptive Gain and the Effect of Volatility

Across temperature profiles of increasing volatility, the adaptive system consistently harvests more energy than the best fixed element, though the margin is modest. The advantage over the best fixed element is approximately 1.3% under smooth conditions, dips slightly at low-to-moderate volatility where a single mid-range element remains a good compromise, and then rises steadily to about 2.0% as the profile becomes more volatile. The advantage over the average fixed element is larger and more stable, around 5% throughout. The interpretation is that volatility is what creates opportunity for adaptivity: the more time the system spends away from any single element's optimum, the more a switching strategy gains.

Table 2: Adaptive energy gain versus operating-profile volatility (averaged over multiple random seeds).

Volatility	**Gain vs best fixed (%)**	**Gain vs average fixed (%)**
0.0	1.63	5.23
0.2	1.35	5.00
0.4	1.33	4.87
0.6	1.59	4.96
0.8	1.89	5.07
1.0	1.98	5.19

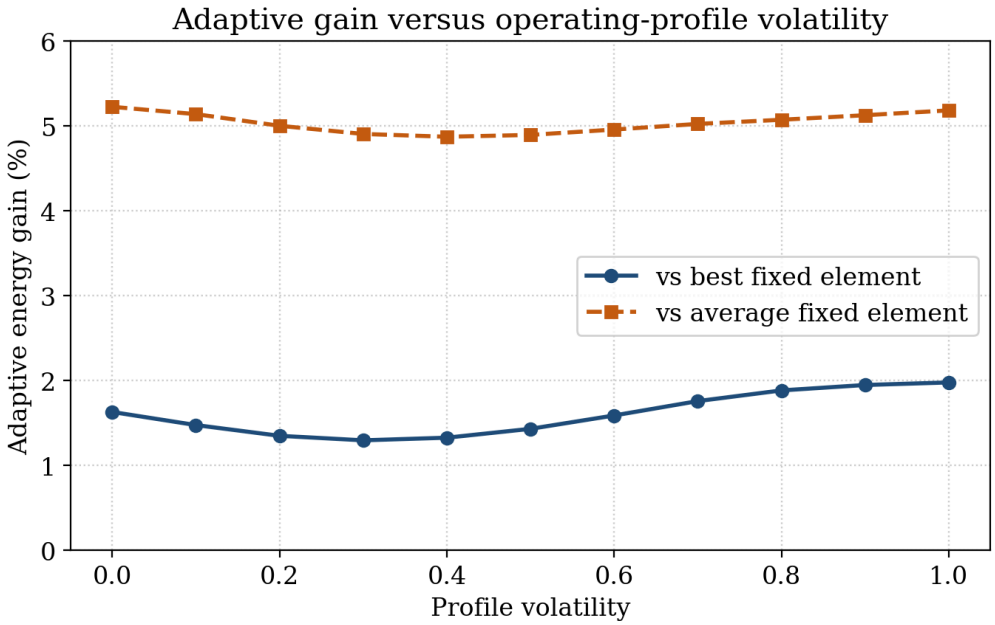


Figure 2: Adaptive energy gain over the best and average fixed element as a function of profile volatility. (Source: volatility_curve.csv / matlab_volatility.png.)

4.3 Robustness Against Unknown Conditions

A more compelling argument for adaptivity emerges from the robustness study. The single best fixed element is not the same across different operating profiles: for cold-biased, mid-range, and hot-biased profiles, one element is best, while for wide or rapidly cycling profiles, a different element is best. In practice, an engineer choosing one fixed element must commit before the operating profile is known, and may therefore choose wrongly. The adaptive system requires no such foreknowledge; it tracks whichever element is currently best and consistently beats the average fixed choice by roughly 5% across every profile tested. Robustness, rather than peak efficiency, is the strongest practical benefit identified in this study.

Table 3: Robustness study. The best fixed element changes with the profile, but adaptive control needs no prior knowledge of it.

Operating profile	**Best fixed element**	**Adaptive vs best (%)**	**Adaptive vs average (%)**
Cold-biased	E2	1.05	4.72
Mid-smooth	E2	1.28	4.86

Wide-volatile	E1	1.76	4.83
Hot-biased	E2	1.18	4.87
Rapid-cycling	E1	2.06	5.23

4.4 The Effect of Switching Cost

The preceding results assume switching is free. In reality, every switch consumes energy. When a per-switch penalty is introduced, the picture changes in an important way. Under the naive controller, which switches whenever any element is momentarily better, the net advantage erodes as cost rises and becomes negative at the highest cost level for moderate volatilities, meaning the adaptive system performs worse than simply using one fixed element. This confirms the concern that switching losses can cancel the modest gains. The cost-aware controller resolves this. By switching only when the expected improvement exceeds the switching cost, it avoids unnecessary switching and remains net positive across all tested cost levels and volatilities. Where the naive controller falls to roughly −0.3% at high cost, the cost-aware controller retains a gain of around 0.7%. The controller design is therefore as important as the element bank: adaptability is only beneficial in practice when the controller is aware of the cost it incurs.

Table 4: Net adaptive gain over the best fixed element after switching cost, for the naive and cost-aware controllers at two cost levels. Negative values indicate adaptivity is no longer worthwhile.

Volatility	**Naive, 0.005**	**Aware, 0.005**	**Naive, 0.010**	**Aware, 0.010**
0.0	1.56	1.37	1.48	1.22
0.2	0.83	1.05	0.33	0.97

0.4	0.49	0.84	−0.32	0.68
0.6	0.60	0.97	−0.37	0.67
0.8	0.89	1.17	−0.07	0.85
1.0	1.00	1.22	0.04	0.76

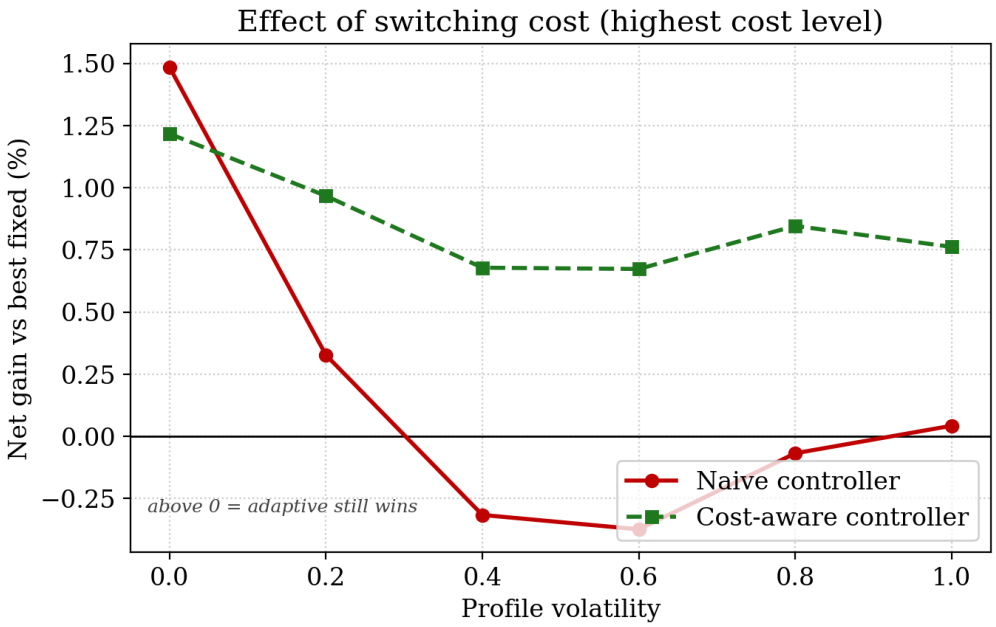


Figure 3: Net adaptive gain versus volatility at the highest switching cost, comparing the naive and cost-aware controllers. (Source: stage3_cost_curve.csv / matlab_cost.png.)

4.5 Synthesis

Taken together, the results support a nuanced conclusion. Adaptive DOS switching does outperform fixed operation, but the raw efficiency gain is small, in the region of one to two percent over the best fixed element. Its value lies less in peak efficiency than in two other properties: the gain grows with the volatility and unpredictability of the operating environment, and adaptivity removes the need to know the operating profile in advance. These benefits are real only when switching is managed by a cost-aware controller, since an unmanaged controller can squander them entirely. The honest engineering

position is therefore that adaptivity is worthwhile for wide, volatile, or unpredictable thermal environments, and unnecessary for stable ones.

5. Conclusion

This study set out to determine whether a thermoelectric system that adapts its electronic density of states to changing temperature can harvest more energy than a fixed material. A computational framework was developed, implemented in C++, and cross-validated against an independent MATLAB model to evaluate an adaptive bank of DOS-engineered elements against fixed alternatives across realistic operating conditions. The framework confirmed that no single element is optimal at all temperatures, providing the basis for adaptive switching. Adaptive selection was found to harvest more energy than the best fixed element consistently, by a modest margin that grows with the volatility of the operating profile, and a larger and more stable margin over the average fixed choice. The most practically significant finding is robustness: because the best fixed element depends on the operating profile, which is rarely known in advance, adaptive control offers a dependable advantage without foreknowledge. Finally, the study showed that switching costs can negate these gains under naive control, but that a cost-aware controller preserves a net benefit across all conditions tested. The principal contribution is not a new material or a large efficiency improvement, but a validated and reusable framework together with a clear, honest characterisation of when adaptive thermoelectric operation is worthwhile. This positions the work as a foundation on which more detailed device-level and materials-level studies can build.

6. Limitations and Future Work

Several limitations should be acknowledged, each pointing to a direction for further research. The model uses an analytic density of states in reduced units rather than a first-principles band structure of a specific material, so it captures trends rather than absolute, material-specific performance. Coupling the framework to density-functional-theory band structures would allow validation against named compounds and experimental data. The switching cost is modelled as a simple per-switch energy penalty, whereas a real implementation would also incur continuous losses, most importantly thermal leakage through inactive elements, which could further reduce or eliminate the modest gains. A fuller thermal model of the element bank is an important next step. The constant relaxation-time approximation, while standard, omits energy-dependent scattering that can itself influence the Seebeck coefficient. The physical mechanism for switching is treated only at the level of a discrete element bank, and the engineering of a practical switch, including its actuation energy, response time, and reliability, remains to be investigated. Finally, the efficiency gains reported here are small enough that their practical value will depend strongly on the specific application and on whether the switching hardware can be made sufficiently cheap and low-loss. These limitations do not undermine the central finding but define the boundary of what the present framework can claim, and they form a concrete agenda for subsequent work.

References

Cutler, M. and Mott, N.F. (1969) ‘Observation of Anderson localization in an electron gas’, Physical Review, 181(3), pp. 1336–1340.
Heremans, J.P., Jovovic, V., Toberer, E.S., Saramat, A., Kurosaki, K., Charoenphakdee, A., Yamanaka, S. and Snyder, G.J. (2008) ‘Enhancement of thermoelectric efficiency in PbTe by distortion of the electronic density of states’, Science, 321(5888), pp. 554–557.
Madsen, G.K.H. and Singh, D.J. (2006) ‘BoltzTraP. A code for calculating band-structure dependent quantities, Computer Physics Communications, 175(1), pp. 67–71.
Mahan, G.D. and Sofo, J.O. (1996) ‘The best thermoelectric’, Proceedings of the National Academy of Sciences, 93(15), pp. 7436–7439.
Snyder, G.J. and Toberer, E.S. (2008) ‘Complex thermoelectric materials’, Nature Materials, 7(2), pp. 105–114.

Appendix A: Source Code Listings

This appendix contains the full MATLAB source code implementation for the computational framework described in this study, as required for this submission. The project's physics-based transport model was originally developed and validated using a high-performance C++ core; the MATLAB listings below serve as the complete, independent implementation of that logic. For clarity and structure, these files are categorized by their functional role within the framework, with specific C++ counterparts noted to illustrate the parallel development and cross-validation approach.

- **Core Transport Engine:** *engine_matlab.m* (MATLAB core physics) - Validated against the high-performance *engine.cpp* numerical core.
- **Switching and Controllers:** *stage2_switching.cpp* (C++ logic implementation) - Defines the element bank and controller algorithms used in the primary simulation.
- **Performance Analysis:** *stage2b_volatility_robustness.cpp* and *stage3_cost_check.cpp* (C++ analytical modules) - Implement the volatility profiling and cost-aware switching metrics.
- **Validation and Utilities:** *switching_matlab.m* (MATLAB utility) - Mirrors the C++ system-level analysis to ensure cross-platform consistency.

1. Core Transport Engine

```
Engine_matlab.m
%
=====================================================
=====
%  Stage 1 Forward Engine  -  MATLAB reference model
%  Adaptive Density-of-States Thermoelectric Project
%
%  PURPOSE
%    Independent re-implementation of the same
transport physics as the
%    C++ core. This is the validation model: if C++ and
MATLAB agree on
%    the same inputs, the computational core is
trustworthy. It also
%    produces the plots for the paper.
%
```

```
%  Physics (same as the C++ engine, restated plainly):
%    - DOS g(E): a baseline parabolic band sqrt(E-edge)
plus an optional
%      Lorentzian resonant peak (the tuning knob).
%    - Transport distribution Sigma(E) = band carriers
+ resonant-peak
%      carriers, under the constant relaxation-time
approximation.
%    - Moments L0,L1,L2 = integral of
(-df/dE)*Sigma(E)*(E-Ef)^n dE.
%    - sigma, S, kappa_e, ZT follow from the moments.
%
%  UNITS: reduced units, kB = 1, e = 1. We validate
trends/shapes, not
%  a specific material's absolute numbers. State this
in the paper.
%
============================================================
=====

clear; clc;

% ---- reduced-unit constants ----
global kB e_charge
kB = 1.0;
e_charge = 1.0;

%
============================================================
=====
%  SANITY CHECK: plain vs tuned element at a realistic
operating point
%
============================================================
=====
Ef = 0.60;   % Fermi level INSIDE the band (degenerate,
heavily doped)
T  = 0.10;   % temperature (reduced units)
kappaLattice = 0.2;

% plain element: baseline band, no resonant peak
plain = struct('bandEdge',0.0,'baseAmp',1.0, ...
```

```
'hasPeak',false,'peakPos',0.0,'peakWidth',0.05,'peakAmp
',0.0);

% tuned element: same band + resonant peak just above
Ef
tuned = plain;
tuned.hasPeak   = true;
tuned.peakPos   = 0.70;
tuned.peakWidth = 0.04;
tuned.peakAmp   = 4.0;

rPlain = computeTransport(plain, Ef, T, kappaLattice);
rTuned = computeTransport(tuned, Ef, T, kappaLattice);

fprintf('=== SANITY CHECK at Ef=%.3f, T=%.3f ===\n\n',
Ef, T);
fprintf('PLAIN element (no resonant peak):\n');
fprintf('  sigma=%.5f  S=%.5f  kappa_e=%.5f  PF=%.5f
ZT=%.5f\n\n', ...
        rPlain.sigma, rPlain.S, rPlain.kappa_e,
rPlain.PF, rPlain.ZT);
fprintf('TUNED element (resonant peak near Ef):\n');
fprintf('  sigma=%.5f  S=%.5f  kappa_e=%.5f  PF=%.5f
ZT=%.5f\n\n', ...
        rTuned.sigma, rTuned.S, rTuned.kappa_e,
rTuned.PF, rTuned.ZT);

%
==========================================================
=====
%  SWEEP 1: peak position (shows the resonance is
position-sensitive)
%
==========================================================
=====
posList = -0.0 + (0.3:0.01:1.0);
S_pos = zeros(size(posList)); PF_pos =
zeros(size(posList));
ZT_pos = zeros(size(posList));
for i = 1:numel(posList)
    s = plain; s.hasPeak = true; s.peakPos =
```

```
posList(i);
    s.peakWidth = 0.04; s.peakAmp = 4.0;
    r = computeTransport(s, Ef, T, kappaLattice);
    S_pos(i) = r.S; PF_pos(i) = r.PF; ZT_pos(i) = r.ZT;
end
T1 = table(posList(:), S_pos(:), PF_pos(:), ZT_pos(:),
...
           'VariableNames', {'peakPos','S','PF','ZT'});
writetable(T1, 'matlab_sweep_peakpos.csv');

%
==========================================================
=====
%  SWEEP 2: ZT vs temperature, plain vs tuned (THE
CROSSOVER)
%  This is the key figure: it shows no single element
wins everywhere,
%  which is the whole justification for adaptive
switching.
%
==========================================================
=====
Tlist = 0.05:0.01:1.0;
ZT_plain = zeros(size(Tlist)); ZT_tuned =
zeros(size(Tlist));
for i = 1:numel(Tlist)
    a = computeTransport(plain, Ef, Tlist(i),
kappaLattice);
    b = computeTransport(tuned, Ef, Tlist(i),
kappaLattice);
    ZT_plain(i) = a.ZT; ZT_tuned(i) = b.ZT;
end
T2 = table(Tlist(:), ZT_plain(:), ZT_tuned(:), ...
           'VariableNames',
{'T','ZT_plain','ZT_tuned'});
writetable(T2, 'matlab_sweep_temp.csv');

% ---- plot the crossover (Figure 1 candidate) ----
figure('Color','w');
plot(Tlist, ZT_plain, 'LineWidth', 2); hold on;
plot(Tlist, ZT_tuned, 'LineWidth', 2);
xlabel('Temperature (reduced units)');
```

```
ylabel('ZT');
legend('Plain element','Tuned (resonant)
element','Location','northwest');
title('ZT vs Temperature: no single element wins
everywhere');
grid on;
saveas(gcf, 'crossover.png');

fprintf('Wrote matlab_sweep_peakpos.csv,
matlab_sweep_temp.csv, crossover.png\n');

%
=============================================================
=====
%  FUNCTIONS
%
=============================================================
=====

function g = dos(E, p)
    % density of states at energy E
    g = 0.0;
    if E > p.bandEdge
        g = g + p.baseAmp * sqrt(E - p.bandEdge);
    end
    if p.hasPeak && p.peakAmp > 0.0
        x = E - p.peakPos;
        lorentz = (p.peakWidth^2) / (x^2 +
p.peakWidth^2);
        g = g + p.peakAmp * lorentz;
    end
end

function w = minusDfDe(E, Ef, T)
    % -df/dE : the transport window weight
    global kB
    kT = kB * T;
    x = (E - Ef) / kT;
    if abs(x) > 40.0
        w = 0.0; return;
    end
    ex = exp(x);
```

```
    denom = (ex + 1.0);
    w = (ex / (denom^2)) / kT;
end

function sigmaE = transportDist(E, p)
    % Sigma(E): band carriers + resonant-peak carriers
(constant-RTA)
    sigmaE = 0.0;
    if E > p.bandEdge
        gBase = p.baseAmp * sqrt(E - p.bandEdge);
        sigmaE = sigmaE + gBase * (E - p.bandEdge);   %
v^2 * tau model term
    end
    if p.hasPeak && p.peakAmp > 0.0
        x = E - p.peakPos;
        lorentz = (p.peakWidth^2) / (x^2 +
p.peakWidth^2);
        gPeak = p.peakAmp * lorentz;
        peakMobility = 0.5;
% resonant-state weight
        sigmaE = sigmaE + gPeak * peakMobility;
    end
end

function m = computeMoments(p, Ef, T)
    % L0,L1,L2 by trapezoidal integration over a wide
energy window
    global kB
    kT = kB * T;
    Emin = min(p.bandEdge, Ef) - 30.0 * kT;
    Emax = max(p.bandEdge, Ef) + 30.0 * kT;
    if p.hasPeak
        Emin = min(Emin, p.peakPos - 10.0 *
p.peakWidth);
        Emax = max(Emax, p.peakPos + 10.0 *
p.peakWidth);
    end

    N  = 200000;
    dE = (Emax - Emin) / N;
    E  = Emin + (0:N) * dE;
```

```
    % vectorized evaluation
    w = arrayfun(@(ee) minusDfDe(ee, Ef, T), E);
    sg = arrayfun(@(ee) transportDist(ee, p), E);
    base = w .* sg;
    dEf  = (E - Ef);

    % trapezoid weights
    tw = ones(1, N+1); tw(1) = 0.5; tw(end) = 0.5;

    L0 = sum(tw .* base) * dE;
    L1 = sum(tw .* base .* dEf) * dE;
    L2 = sum(tw .* base .* dEf.^2) * dE;
    m = struct('L0',L0,'L1',L1,'L2',L2);
end

function r = computeTransport(p, Ef, T, kappaLattice)
    global e_charge
    m = computeMoments(p, Ef, T);
    r =
struct('sigma',0,'S',0,'kappa_e',0,'kappa_total',kappaL
attice, ...
               'ZT',0,'PF',0,'powerProxy',0);
    if m.L0 <= 1e-300
        return;
    end
    r.sigma    = e_charge^2 * m.L0;
    r.S        = (1.0 / (e_charge * T)) * (m.L1 / m.L0);
    r.kappa_e = (1.0 / T) * (m.L2 - (m.L1^2) / m.L0);
    if r.kappa_e < 0, r.kappa_e = 0; end
    r.kappa_total = r.kappa_e + kappaLattice;
    r.PF          = r.S^2 * r.sigma;
    r.ZT          = r.PF * T / r.kappa_total;
    r.powerProxy  = r.PF * T^2;
end
```

4. Validation and Utilities

Switching_matlab.m

```
%
```

```
%% ========================================================
% Stage 2/2b/3 - MATLAB reference model
% Adaptive Density-of-States Thermoelectric Project
%
% PURPOSE
%   Independent MATLAB re-implementation of the switching framework
%   (element bank, controllers) and the cost study, mirroring the C++
%   core. Running this and matching the C++ reference table validates
%   the WHOLE framework, not just the single-element engine.
%
% Physics identical to Stage 1 (constant-RTA Boltzmann, reduced units
% kB = e = 1). Comparison metric is efficiency-based harvested power.
%
% This script:
%   1) prints a VALIDATION table (match against the C++ probe)
%   2) runs the volatility study  -> matlab_volatility_curve.csv + plot
%   3) runs the cost study (naive vs cost-aware) -> matlab_cost_curve.csv + plot
%
% ========================================================
clear; clc;
global kB e_charge
kB = 1.0; e_charge = 1.0;
bank  = buildBank();
steps = 400;
%
% ========================================================
% 1. VALIDATION TABLE  (match these against the C++ stage-2 probe)
%    Single-element ZT at fixed (element, T) points.
%
```

```
=============================================================
=====
fprintf('=== VALIDATION TABLE (match against C++)
===\n');
testT = [0.10, 0.30, 0.50];
for k = 1:numel(testT)
    T = testT(k);
    fprintf('T=%.2f : ', T);
    for i = 1:numel(bank)
        zt = computeZT(bank(i).dos, bank(i).Ef, T);
        fprintf('%s ZT=%.5f  ', bank(i).name, zt);
    end
    fprintf('\n');
end
fprintf('\n');
%
=============================================================
=====
%  2. VOLATILITY STUDY
%
=============================================================
=====
fprintf('=== VOLATILITY STUDY ===\n');
fprintf('vol   gain_vs_best%%   gain_vs_avg%%\n');
vols = 0.0:0.2:1.0;
gBest = zeros(size(vols)); gAvg = zeros(size(vols));
reps = 4;
for v = 1:numel(vols)
    accB = 0; accA = 0;
    for r = 1:reps
        prof = makeProfile(vols(v), steps, 100+r);
        [aE, fE] = runProfile(bank, prof);
        bestFixed = max(fE);
        avgFixed  = mean(fE);
        accB = accB + 100*(aE-bestFixed)/bestFixed;
        accA = accA + 100*(aE-avgFixed)/avgFixed;
    end
    gBest(v) = accB/reps; gAvg(v) = accA/reps;
    fprintf('%.1f   %8.3f       %8.3f\n', vols(v),
gBest(v), gAvg(v));
end
writetable(table(vols(:),gBest(:),gAvg(:), ...
```

```
'VariableNames',{'volatility','gain_vs_best_pct','gain_
vs_avg_pct'}), ...
   'matlab_volatility_curve.csv');
figure('Color','w');
plot(vols, gBest, '-o', 'LineWidth', 2); hold on;
plot(vols, gAvg, '-s', 'LineWidth', 2);
xlabel('Profile volatility'); ylabel('Adaptive energy
gain (%)');
legend('vs best fixed','vs average
fixed','Location','northwest');
title('Adaptive gain grows with profile volatility');
grid on;
saveas(gcf, 'matlab_volatility.png');
%
==========================================================
=====
%  3. COST STUDY : naive vs cost-aware controller after
switching cost
%
==========================================================
=====
fprintf('\n=== COST STUDY (net gain%% vs best fixed,
N=naive A=aware) ===\n');
costLevels = [0.0, 0.002, 0.005, 0.010];
header = 'vol';
for c = 1:numel(costLevels), header = [header sprintf('
cost=%.3f N/A',costLevels(c))]; end
disp(header);
results = [];
for v = 1:numel(vols)
   row = vols(v); line = sprintf('%.1f', vols(v));
   for c = 1:numel(costLevels)
       accN = 0; accA = 0;
       for r = 1:reps
           prof = makeProfile(vols(v), steps, 200+r);
           sd = precompute(bank, prof);
           bestFixed = runBestFixed(sd, numel(bank));
           netN = runNaive(sd, costLevels(c));
           netA = runCostAware(sd, costLevels(c));
           accN = accN +
100*(netN-bestFixed)/bestFixed;
```

```
            accA = accA +
100*(netA-bestFixed)/bestFixed;
        end
        accN = accN/reps; accA = accA/reps;
        row = [row accN accA];
        line = [line sprintf('   %5.2f/%5.2f', accN,
accA)];
    end
    results = [results; row];
    disp(line);
end
% save cost curve
vn = {'volatility'};
for c = 1:numel(costLevels)
    vn{end+1} = sprintf('naive_c%.3f', costLevels(c));
    vn{end+1} = sprintf('aware_c%.3f', costLevels(c));
end
writetable(array2table(results,'VariableNames',vn),
'matlab_cost_curve.csv');
% plot the highest-cost case: naive vs aware across
volatility
figure('Color','w');
naiveHi = results(:, end-1); awareHi = results(:, end);
plot(vols, naiveHi, '-o', 'LineWidth', 2); hold on;
plot(vols, awareHi, '-s', 'LineWidth', 2);
yline(0, '--k');
xlabel('Profile volatility'); ylabel('Net gain vs best
fixed (%)');
legend('Naive controller','Cost-aware
controller','Location','best');
title(sprintf('Does adaptive survive switching cost?
(cost=%.3f)', costLevels(end)));
grid on;
saveas(gcf, 'matlab_cost.png');
fprintf('\nWrote matlab_volatility_curve.csv,
matlab_cost_curve.csv, and PNGs.\n');
%
=============================================================
=====
%  FUNCTIONS
%
=============================================================
```

```
=====
function bank = buildBank()
    centers = [0.08, 0.18, 0.32, 0.50, 0.72];
    bank = struct('name',{},'dos',{},'Ef',{});
    for i = 1:numel(centers)
        d =
struct('bandEdge',0,'baseAmp',1,'hasPeak',true, ...

'peakPos',centers(i)+0.05,'peakWidth',0.03,'peakAmp',5.
0);
        bank(i) =
struct('name',sprintf('E%d',i-1),'dos',d,'Ef',centers(i
));
    end
end
function w = minusDfDe(E, Ef, T)
    global kB
    kT = kB*T; x = (E-Ef)/kT;
    if abs(x) > 40, w = 0; return; end
    ex = exp(x); d = ex+1; w = (ex/(d^2))/kT;
end
function s = transportDist(E, p)
    s = 0;
    if E > p.bandEdge
        g = p.baseAmp*sqrt(E-p.bandEdge); s = s +
g*(E-p.bandEdge);
    end
    if p.hasPeak && p.peakAmp > 0
        x = E-p.peakPos; l =
(p.peakWidth^2)/(x^2+p.peakWidth^2);
        s = s + p.peakAmp*l*0.5;
    end
end
function zt = computeZT(p, Ef, T)
    global kB e_charge
    kL = 0.2; kT = kB*T;
    Emin = min(p.bandEdge,Ef)-30*kT; Emax =
max(p.bandEdge,Ef)+30*kT;
    if p.hasPeak
        Emin = min(Emin, p.peakPos-10*p.peakWidth);
        Emax = max(Emax, p.peakPos+10*p.peakWidth);
    end
```

```
    N = 8000; dE = (Emax-Emin)/N; E = Emin + (0:N)*dE;
    w  = arrayfun(@(e) minusDfDe(e,Ef,T), E);
    sg = arrayfun(@(e) transportDist(e,p), E);
    b = w.*sg; d = E-Ef;
    tw = ones(1,N+1); tw(1)=0.5; tw(end)=0.5;
    L0 = sum(tw.*b)*dE; L1 = sum(tw.*b.*d)*dE; L2 =
sum(tw.*b.*d.^2)*dE;
    if L0 <= 1e-300, zt = 0; return; end
    S = (1/(e_charge*T))*(L1/L0); sigma = e_charge^2*L0;
    ke = (1/T)*(L2 - L1^2/L0); if ke<0, ke=0; end
    PF = S^2*sigma; zt = PF*T/(ke+kL);
end
function h = harvested(zt), h = zt/(zt+2.0); end
function prof = makeProfile(volatility, steps, seed)
    rng(seed);
    prof = zeros(1, steps); T = 0.4;
    for t = 1:steps
        smooth = 0.4 +
(0.30*(1-volatility))*sin(2*pi*(t-1)/200);
        T = 0.7*T + 0.3*smooth +
(0.25*volatility)*randn;
        if T<0.03, T=0.03; end
        if T>0.95, T=0.95; end
        prof(t) = T;
    end
end
function [adaptiveE, fixedE] = runProfile(bank, prof)
    n = numel(bank); fixedE = zeros(1,n); adaptiveE = 0;
    for t = 1:numel(prof)
        h = zeros(1,n);
        for i = 1:n, h(i) =
harvested(computeZT(bank(i).dos, bank(i).Ef, prof(t)));
end
        fixedE = fixedE + h;
        adaptiveE = adaptiveE + max(h);
    end
end
function sd = precompute(bank, prof)
    n = numel(bank); sd = zeros(numel(prof), n);
    for t = 1:numel(prof)
        for i = 1:n, sd(t,i) =
harvested(computeZT(bank(i).dos, bank(i).Ef, prof(t)));
```

```
end
    end
end
function net = runNaive(sd, switchCost)
    net = 0; prev = -1;
    for t = 1:size(sd,1)
        [~, best] = max(sd(t,:)); best = best-1;
        net = net + sd(t,best+1);
        if prev ~= -1 && best ~= prev, net = net -
switchCost; end
        prev = best;
    end
end
function net = runCostAware(sd, switchCost)
    net = 0; cur = -1;
    for t = 1:size(sd,1)
        [~, best] = max(sd(t,:)); best = best-1;
        if cur == -1
            cur = best; net = net + sd(t,cur+1);
continue;
        end
        if best ~= cur && (sd(t,best+1) - sd(t,cur+1)) >
switchCost
            net = net + sd(t,best+1) - switchCost; cur =
best;
        else
            net = net + sd(t,cur+1);
        end
    end
end
function bf = runBestFixed(sd, nEl)
    tot = sum(sd(:,1:nEl), 1);
    bf = max(tot);
end
```